COMPILER CONSTRUCTION LAB

Lab Manual

Top-Down Parsing: Non-Recursive Descent Parser (LL(1) Parser)

Course Title	Compiler Construction
Lab Number	Lab Session — LL(1) Parser
Programme	BS Computer Science (Undergraduate)
Duration	3 Hours
Pre-requisites	Theory of Automata, Recursive Descent Parsing

1. Lab Objectives

After successfully completing this lab, the student will be able to:

1. Differentiate between recursive descent parsing and non-recursive (table driven) LL(1) parsing.
2. Compute FIRST and FOLLOW sets for any context-free grammar.
3. Construct an LL(1) predictive parsing table from a given grammar.
4. Determine whether a given grammar is LL(1) by detecting conflicts in the parsing table.
5. Implement a non-recursive predictive parser using a stack and parsing table in C/C++.
6. Trace the parsing actions for a sample input string and report acceptance or rejection.

3. Background Theory

3.1 Top-Down Parsing

Top-down parsing is a strategy in which the parser begins with the start symbol of the grammar and attempts to derive the input string by repeatedly replacing non-terminals with the right-hand side of one of their productions. The parser builds the parse tree from the root and grows it downwards toward the leaves, which represent the terminals of the input. The parser succeeds when the entire input is consumed and the derivation is complete.

Top-down parsers come in two main flavours: recursive descent parsers and non-recursive (table driven) parsers. Both are predictive in nature when the underlying grammar is LL(1), but they differ in implementation strategy.

3.2 Recursive vs Non-Recursive Descent

In a recursive descent parser, every non-terminal of the grammar is implemented as a separate function, and the parser uses the program call stack implicitly through recursion. While easy to write by hand, this approach is tightly coupled with the grammar and can be inefficient for large grammars.

A non-recursive (table driven) parser eliminates recursion by using an explicit stack and a two dimensional parsing table. The same parser engine can drive any LL(1) grammar simply by supplying a different table. This makes the approach more general purpose, easier to automate, and a natural fit for compiler construction tools.

3.3 The LL(1) Parser

The name LL(1) is read as follows: the first L means the input is scanned from Left to right, the second L means a Leftmost derivation is produced, and the digit 1 means the parser uses one symbol of look-ahead to decide which production to apply. An LL(1) parser is therefore a predictive parser that never needs to backtrack.

A grammar is said to be LL(1) if and only if its predictive parsing table contains no entry with more than one production. Practically, this requires that the grammar be free of left recursion and properly left factored.

3.4 Components of a Non-Recursive LL(1) Parser

The non-recursive predictive parser is built from four cooperating components, as illustrated conceptually below:

- Input Buffer: holds the string to be parsed, terminated by an end marker $.
- Stack: holds a sequence of grammar symbols. Initially the stack contains $ at the bottom and the start symbol on top.
- Parsing Table M: a two dimensional array M[A, a] indexed by a non-terminal A and a terminal a, returning the production that should be applied.
- Output: the sequence of productions used, which represents a leftmost derivation of the input.

3.5 FIRST and FOLLOW Sets

Construction of the parsing table requires two auxiliary sets called FIRST and FOLLOW. These sets formalise the look-ahead reasoning of the parser.

Rules for computing FIRST(α)

7. If X is a terminal then FIRST(X) = { X }.
8. If X → ε is a production then add ε to FIRST(X).
9. If X is a non-terminal and X → Y1 Y2 … Yk, then place a in FIRST(X) if for some i, a is in FIRST(Yi) and ε is in FIRST(Y1)…FIRST(Yi-1). If ε is in FIRST(Yj) for all j = 1..k, then add ε to FIRST(X).

Rules for computing FOLLOW(A)

10. Place $ in FOLLOW(S) where S is the start symbol and $ is the input end marker.
11. If there is a production A → α B β then everything in FIRST(β) except ε is placed in FOLLOW(B).
12. If there is a production A → α B, or A → α B β where ε is in FIRST(β), then everything in FOLLOW(A) is in FOLLOW(B).

3.6 Construction of the Parsing Table

Given the FIRST and FOLLOW sets, the predictive parsing table M is built using the following algorithm. For each production A → α of the grammar:

13. For each terminal a in FIRST(α), add A → α to M[A, a].
14. If ε is in FIRST(α), then for each terminal b in FOLLOW(A), add A → α to M[A, b].
15. All entries that remain undefined after this process are designated as error entries.

If any cell of the table is assigned more than one production, the grammar is not LL(1) and a non-recursive predictive parser cannot be built directly without further transformation (left factoring or removal of left recursion).

3.7 The Parsing Algorithm

The non-recursive predictive parser repeatedly inspects the symbol on top of the stack X and the current input symbol a, and performs one of the following actions:

16. If X = a = $, the parser halts and announces successful parsing.
17. If X = a (terminal match), pop X from the stack and advance the input pointer.
18. If X is a non-terminal, consult M[X, a]. If the entry is a production X → Y1 Y2 … Yk, pop X from the stack and push Yk, Yk-1, …, Y1 on the stack so that Y1 is on top. Output the production used.
19. Otherwise (entry is error or X is a terminal not equal to a), invoke an error recovery routine.

4. Worked Example

4.1 The Grammar

Consider the classical expression grammar after removal of left recursion:

```
E  -> T E'
E' -> + T E' | epsilon
T  -> F T'
T' -> * F T' | epsilon
F  -> ( E ) | id
```

4.2 FIRST and FOLLOW Sets

The FIRST and FOLLOW sets for this grammar are:

Non-Terminal	FIRST	FOLLOW
E	{ (, id }	{) , $ }
E'	{ + , epsilon }	{) , $ }
T	{ (, id }	{ + ,) , $ }
T'	{ * , epsilon }	{ + ,) , $ }
F	{ (, id }	{ + , * ,) , $ }

4.3 The LL(1) Parsing Table

Applying the construction rules yields the following parsing table. Empty cells represent error entries.

	id	+	*	(	)	$
E	E -> T E'			E -> T E'		
E'		E' -> + T E'			E' -> e	E' -> e
T	T -> F T'			T -> F T'		
T'		T' -> e	T' -> * F T'		T' -> e	T' -> e
F	F -> id			F -> (E)		

Note: In this table, the symbol e is used as a shorthand for epsilon (the empty production).

4.4 Trace for Input id + id * id

The table below shows a step by step trace of the parser for the input id + id * id. The right hand column lists the action taken at each step.

Stack	Input	Output	Action

$ E	id + id * id $		Initial
$ E' T	id + id * id $	E -> T E'	Expand
$ E' T' F	id + id * id $	T -> F T'	Expand
$ E' T' id	id + id * id $	F -> id	Expand
$ E' T'	+ id * id $		Match id
$ E'	+ id * id $	T' -> e	Expand
$ E' T +	+ id * id $	E' -> + T E'	Expand
$ E' T	id * id $		Match +
$ E' T' F	id * id $	T -> F T'	Expand
$ E' T' id	id * id $	F -> id	Expand
$ E' T'	* id $		Match id
$ E' T' F *	* id $	T' -> * F T'	Expand
$ E' T' F	id $		Match *
$ E' T' id	id $	F -> id	Expand
$ E' T'	$		Match id
$ E'	$	T' -> e	Expand
$	$	E' -> e	Expand
$	$		**ACCEPT**

5. Sample Implementation in C

The following C program implements the non-recursive predictive parser for the expression grammar shown above. The parsing table is hard coded as a two dimensional array of strings, but the same engine works for any LL(1) grammar.

5.1 Source Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define STACK_SIZE 100

/* Non-terminals: E=0, E'=1, T=2, T'=3, F=4 */
/* Terminals    : id=0, +=1, *=2, (=3, )=4, $=5  */

char *table[5][6] = {
    /*           id        +          *          (          )       $   */
    /* E  */  { "TE'",     "",        "",        "TE'",    "",    ""  },
    /* E' */  { "",        "+TE'",    "",        "",       "e",   "e" },
    /* T  */  { "FT'",     "",        "",        "FT'",    "",    ""  },
    /* T' */  { "",        "e",       "*FT'",    "",       "e",   "e" },
    /* F  */  { "i",       "",        "",        "(E)",    "",    ""  }
};

char stack[STACK_SIZE];
int  top = -1;

void push(char c) { stack[++top] = c; }
char pop()        { return stack[top--]; }

int ntIndex(char c) {
    switch (c) {
        case 'E': return 0;
        case 'e': return 1;   /* using lowercase e for E' */
        case 'T': return 2;
        case 't': return 3;   /* using lowercase t for T' */
        case 'F': return 4;
        default : return -1;
    }
}

int tIndex(char c) {
    switch (c) {
        case 'i': return 0;
        case '+': return 1;
        case '*': return 2;
        case '(': return 3;
        case ')': return 4;
```

```
        case '$': return 5;
        default : return -1;
    }
}

int isNonTerminal(char c) { return ntIndex(c) != -1; }

int main() {
    char input[100];
    printf("Enter input string ending with $ (use i for id): ");
    scanf("%s", input);

    push('$');
    push('E');

    int ip = 0;
    printf("\n%-25s%-25s%s\n", "STACK", "INPUT", "ACTION");

    while (top >= 0) {
        char X = stack[top];
        char a = input[ip];

        printf("%-25.*s%-25s", top + 1, stack, input + ip);

        if (X == a) {
            if (X == '$') { printf("ACCEPT\n"); return 0; }
            printf("Match %c\n", X);
            pop();
            ip++;
        } else if (isNonTerminal(X)) {
            char *prod = table[ntIndex(X)][tIndex(a)];
            if (strlen(prod) == 0) { printf("ERROR\n"); return 1; }
            printf("%c -> %s\n", X, prod);
            pop();
            if (strcmp(prod, "e") != 0) {
                for (int i = strlen(prod) - 1; i >= 0; i--)
                    push(prod[i]);
            }
        } else {
            printf("ERROR\n");
            return 1;
        }
    }
    return 0;
}
```

5.2 Sample Run

Compile and run the program. A typical interaction is shown below:

```
Enter input string ending with $ (use i for id): i+i*i$
```

STACK	INPUT	ACTION
$E	i+i*i$	E -> TE'
$E'T	i+i*i$	T -> FT'
$E'T'F	i+i*i$	F -> i
$E'T'i	i+i*i$	Match i
$E'T'	+i*i$	T' -> e
$E'	+i*i$	E' -> +TE'
$E'T+	+i*i$	Match +
...	...	...
$	$	ACCEPT

6. Lab Procedure

Carry out the following steps in sequence and record your observations after each step.

20. Read and understand the theory section, especially the rules for FIRST, FOLLOW and parsing table construction.
21. On paper, compute the FIRST and FOLLOW sets for the grammar provided in the worked example and verify that they match Table 4.2.
22. Construct the LL(1) parsing table on paper and compare it with Table 4.3.
23. Open the development environment and create a new C source file named ll1_parser.c.
24. Type the program from Section 5.1, save it, and compile it using your compiler.
25. Execute the program and test it with at least three valid input strings and two invalid ones.
26. Modify the program so that it also prints the leftmost derivation and the parse tree (textual form).
27. Demonstrate the working program to your lab instructor and answer the post-lab questions.

8. Post-Lab Questions

28. State two advantages and two disadvantages of a non-recursive predictive parser when compared with a recursive descent parser.
29. Why must a grammar be free of left recursion before an LL(1) parsing table can be constructed?
30. Explain the role of the FOLLOW set in handling productions that derive epsilon.
31. Give an example of a grammar that is unambiguous but still not LL(1).
32. If a parsing table cell contains two different productions, what does that imply about the grammar and how can it sometimes be remedied?

9. References

33. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques and Tools, 2nd ed., Pearson, 2007.
34. K. C. Louden, Compiler Construction: Principles and Practice, PWS Publishing, 1997.
35. D. Grune and C. J. H. Jacobs, Parsing Techniques: A Practical Guide, 2nd ed., Springer, 2008.
36. R. Mak, Writing Compilers and Interpreters: A Modern Software Engineering Approach, Wiley, 2009.